

## ЛАБОРАТОРНАЯ РАБОТА №3

**Тема:** Реализация структур данных различными способами и сравнение их производительности

**Цель работы:** Изучить способы реализации структур данных через массив, связанный список и стандартную библиотеку языка C++, а также сравнить их производительность и удобство использования.

### Постановка задачи:

Реализовать программу, которая анализирует последовательность пар символов.

Каждая пара символов определяет отношение порядка между ними. Например, пара (b,c) означает, что символ b предшествует символу c.

Из отношений:

- (b,c)
- (c,a)

следует отношение:

- (b,a)

Необходимо определить:

1. Является ли последовательность полной, то есть можно ли все символы выстроить в одну цепочку порядка.
2. Является ли последовательность противоречивой, то есть существует ли ситуация, когда одновременно можно получить:

- (x,y)
- (y,x)

Структуры данных необходимо реализовать тремя способами:

1. Через массив
2. Через связанный список
3. С использованием стандартной библиотеки STL

Также необходимо сравнить производительность каждой реализации.

### Теоретические сведения

#### Массив

Массив представляет собой набор элементов фиксированного размера, расположенных в памяти последовательно. В данной работе массив использовался для создания матрицы отношений между символами.

Преимущества:

- высокая скорость доступа;
- простота реализации.

Недостатки:

- фиксированный размер;
- возможный перерасход памяти.

### **Связанный список**

Связанный список состоит из узлов, каждый из которых хранит данные и указатель на следующий элемент.

Преимущества:

- динамическое выделение памяти;
- удобное добавление элементов.

Недостатки:

- более сложная реализация;
- меньшая скорость доступа.

### **STL**

STL — стандартная библиотека шаблонов языка C++. В работе использовался контейнер `vector`.

Преимущества:

- простота использования;
- высокая надежность;
- удобство работы.

Недостатки:

- меньше контроля над внутренней реализацией.

### **Описание алгоритма**

Программа выполняет следующие действия:

1. Пользователь вводит количество пар символов.
2. Вводятся пары символов.
3. Формируется матрица отношений.
4. Выполняется поиск транзитивных отношений.
5. Проверяется наличие противоречий.
6. Проверяется полнота последовательности.
7. Измеряется время выполнения программы.

Для поиска косвенных отношений использовался алгоритм транзитивного замыкания.

## Реализация программы

### Реализация через массив

В данной реализации использовалась двумерная матрица размером 26x26.

Каждая ячейка матрицы показывает наличие отношения между символами.

Пример: `matrix[a - 'a'][b - 'a'] = 1;`

### Реализация через связанный список

Был создан собственный односвязный список.

Структура узла: `struct Node`

```
{  
    char from;  
    char to;  
    Node* next;  
};
```

### Реализация через STL

В данной реализации использовался контейнер: `vector<pair<char, char>>`

Контейнер хранит пары символов и позволяет удобно работать с данными.

### Листинг:

```
#include <iostream>  
#include <vector>  
#include <chrono>
```

```

using namespace std;
void printAuthor()
{
    cout << "ФИО: Хилобок Андрей Викторович" << endl;
    cout << "Группа: РПИа-о25" << endl;
    cout << endl;
}
void arrayImplementation()
{
    cout << "===== РЕАЛИЗАЦИЯ ЧЕРЕЗ МАССИВ =====" << endl;
    int n;
    cout << "Введите количество пар: ";
    cin >> n;
    int matrix[26][26] = {0};
    vector<char> symbols;
    auto start = chrono::high_resolution_clock::now();
    for (int i = 0; i < n; i++)
    {
        char a, b;
        cout << "Введите пару: ";
        cin >> a >> b;
        matrix[a - 'a'][b - 'a'] = 1;
        bool foundA = false;
        bool foundB = false;
        for (char c : symbols)
        {
            if (c == a)
                foundA = true;

            if (c == b)
                foundB = true;
        }
        if (!foundA)
            symbols.push_back(a);

        if (!foundB)
            symbols.push_back(b);
    }
    // Транзитивность
    for (int k = 0; k < 26; k++)
    {
        for (int i = 0; i < 26; i++)
        {
            for (int j = 0; j < 26; j++)
            {
                if (matrix[i][k] && matrix[k][j])
                {
                    matrix[i][j] = 1;
                }
            }
        }
    }
    bool contradiction = false;
    for (int i = 0; i < 26; i++)
    {
        if (matrix[i][i])
        {
            contradiction = true;
        }
        for (int j = 0; j < 26; j++)
        {
            if (matrix[i][j] && matrix[j][i] && i != j)
            {
                contradiction = true;
            }
        }
    }
}

```

```

    }
}
bool complete = true;
for (int i = 0; i < symbols.size(); i++)
{
    for (int j = i + 1; j < symbols.size(); j++)
    {
        int a = symbols[i] - 'a';
        int b = symbols[j] - 'a';

        if (!matrix[a][b] && !matrix[b][a])
        {
            complete = false;
        }
    }
}

auto end = chrono::high_resolution_clock::now();
auto duration = chrono::duration_cast<chrono::microseconds>(end - start);
if (contradiction)
{
    cout << "Последовательность противоречива" << endl;
}
else
{
    cout << "Противоречий нет" << endl;
}
if (complete)
{
    cout << "Последовательность полная" << endl;
}
else
{
    cout << "Последовательность не полная" << endl;
}
cout << "Время работы: " << duration.count() << " мкс" << endl;
cout << endl;
}

struct Node
{
    char from;
    char to;
    Node* next;
};

void linkedListImplementation()
{
    cout << "===== РЕАЛИЗАЦИЯ ЧЕРЕЗ СВЯЗАННЫЙ СПИСОК =====" << endl;
    int n;
    cout << "Введите количество пар: ";
    cin >> n;
    Node* head = nullptr;
    vector<char> symbols;
    int matrix[26][26] = {0};
    auto start = chrono::high_resolution_clock::now();
    for (int i = 0; i < n; i++)
    {
        char a, b;
        cout << "Введите пару: ";
        cin >> a >> b;
        Node* newNode = new Node;
        newNode->from = a;
        newNode->to = b;
        newNode->next = head;
    }
}

```

```

head = newNode;
matrix[a - 'a'][b - 'a'] = 1;
bool foundA = false;
bool foundB = false;
for (char c : symbols)
{
    if (c == a)
        foundA = true;

    if (c == b)
        foundB = true;
}
if (!foundA)
    symbols.push_back(a);

if (!foundB)
    symbols.push_back(b);
}
for (int k = 0; k < 26; k++)
{
    for (int i = 0; i < 26; i++)
    {
        for (int j = 0; j < 26; j++)
        {
            if (matrix[i][k] && matrix[k][j])
            {
                matrix[i][j] = 1;
            }
        }
    }
}
bool contradiction = false;
for (int i = 0; i < 26; i++)
{
    if (matrix[i][i])
    {
        contradiction = true;
    }
    for (int j = 0; j < 26; j++)
    {
        if (matrix[i][j] && matrix[j][i] && i != j)
        {
            contradiction = true;
        }
    }
}
bool complete = true;
for (int i = 0; i < symbols.size(); i++)
{
    for (int j = i + 1; j < symbols.size(); j++)
    {
        int a = symbols[i] - 'a';
        int b = symbols[j] - 'a';

        if (!matrix[a][b] && !matrix[b][a])
        {
            complete = false;
        }
    }
}
auto end = chrono::high_resolution_clock::now();
auto duration = chrono::duration_cast<chrono::microseconds>(end - start);
if (contradiction)
{

```

```

        cout << "Последовательность противоречива" << endl;
    }
    else
    {
        cout << "Противоречий нет" << endl;
    }
    if (complete)
    {
        cout << "Последовательность полная" << endl;
    }
    else
    {
        cout << "Последовательность не полная" << endl;
    }
    cout << "Время работы: " << duration.count() << " мкс" << endl;
    cout << endl;
    while (head != nullptr)
    {
        Node* temp = head;
        head = head->next;
        delete temp;
    }
}

void stlImplementation()
{
    cout << "===== РЕАЛИЗАЦИЯ ЧЕРЕЗ STL =====" << endl;
    int n;
    cout << "Введите количество пар: ";
    cin >> n;
    vector<pair<char, char>> pairs;
    int matrix[26][26] = {0};
    vector<char> symbols;
    auto start = chrono::high_resolution_clock::now();
    for (int i = 0; i < n; i++)
    {
        char a, b;
        cout << "Введите пару: ";
        cin >> a >> b;
        pairs.push_back(make_pair(a, b));
        matrix[a - 'a'][b - 'a'] = 1;
        bool foundA = false;
        bool foundB = false;
        for (char c : symbols)
        {
            if (c == a)
                foundA = true;

            if (c == b)
                foundB = true;
        }
        if (!foundA)
            symbols.push_back(a);

        if (!foundB)
            symbols.push_back(b);
    }
    for (int k = 0; k < 26; k++)
    {
        for (int i = 0; i < 26; i++)
        {
            for (int j = 0; j < 26; j++)
            {
                if (matrix[i][k] && matrix[k][j])

```

```

        {
            matrix[i][j] = 1;
        }
    }
}

bool contradiction = false;
for (int i = 0; i < 26; i++)
{
    if (matrix[i][i])
    {
        contradiction = true;
    }

    for (int j = 0; j < 26; j++)
    {
        if (matrix[i][j] && matrix[j][i] && i != j)
        {
            contradiction = true;
        }
    }
}

bool complete = true;
for (int i = 0; i < symbols.size(); i++)
{
    for (int j = i + 1; j < symbols.size(); j++)
    {
        int a = symbols[i] - 'a';
        int b = symbols[j] - 'a';

        if (!matrix[a][b] && !matrix[b][a])
        {
            complete = false;
        }
    }
}

auto end = chrono::high_resolution_clock::now();
auto duration = chrono::duration_cast<chrono::microseconds>(end - start);
if (contradiction)
{
    cout << "Последовательность противоречива" << endl;
}
else
{
    cout << "Противоречий нет" << endl;
}
if (complete)
{
    cout << "Последовательность полная" << endl;
}
else
{
    cout << "Последовательность не полная" << endl;
}
cout << "Время работы: " << duration.count() << " мкс" << endl;
cout << endl;
}

int main()
{
    setlocale(LC_ALL, "Russian");
    printAuthor();
    arrayImplementation();
    linkedListImplementation();
}

```



```
    stlImplementation();  
    return 0;  
}
```

## Результаты работы программы

### Пример противоречий нет, последовательность полная:

ФИО: Хилобок Андрей Викторович

Группа: РПИа-о25

===== РЕАЛИЗАЦИЯ ЧЕРЕЗ МАССИВ =====

Введите количество пар: 3

Введите пару: b c

Введите пару: c a

Введите пару: a d

Противоречий нет

Последовательность полная

Время работы: 5283999 мкс

===== РЕАЛИЗАЦИЯ ЧЕРЕЗ СВЯЗАННЫЙ СПИСОК =====

Введите количество пар: 3

Введите пару: b c

Введите пару: c a

Введите пару: a d

Противоречий нет

Последовательность полная

Время работы: 9129190 мкс

===== РЕАЛИЗАЦИЯ ЧЕРЕЗ STL =====

Введите количество пар: 3

Введите пару: b c

Введите пару: c a

Введите пару: a d

Противоречий нет

Последовательность полная

Время работы: 8046215 мкс

### Пример противоречивой последовательности:

ФИО: Хилобок Андрей Викторович

Группа: РПИа-о25

===== РЕАЛИЗАЦИЯ ЧЕРЕЗ МАССИВ =====

Введите количество пар: 3

Введите пару: a b

Введите пару: b a

Введите пару: c d

Последовательность противоречива

Последовательность не полная

Время работы: 15261451 мкс

===== РЕАЛИЗАЦИЯ ЧЕРЕЗ СВЯЗАННЫЙ СПИСОК =====

Введите количество пар: 3

Введите пару: a b

Введите пару: b a

Введите пару: c d

Последовательность противоречива

Последовательность не полная

Время работы: 6432271 мкс

===== РЕАЛИЗАЦИЯ ЧЕРЕЗ STL =====

Введите количество пар: 3

Введите пару: a b

Введите пару: b a

Введите пару: c d

Последовательность противоречива

Последовательность не полная

Время работы: 7798881 мкс

## Сравнение реализаций

| Реализация       | Скорость работы | Сложность реализации | Удобство |
|------------------|-----------------|----------------------|----------|
| Массив           | Высокая         | Простая              | Среднее  |
| Связанный список | Средняя         | Средняя              | Среднее  |
| STL              | Высокая         | Простая              | Высокое  |

**Вывод:** В ходе лабораторной работы были реализованы структуры данных тремя различными способами: через массив, связанный список и стандартную библиотеку STL. Все реализации успешно выполняют проверку полноты и противоречивости последовательности символов. Наиболее простой и удобной оказалась реализация с использованием STL. Реализация через массив показала высокую скорость работы. Реализация через связанный список оказалась более сложной, однако позволила изучить динамические структуры данных. В ходе выполнения работы были получены навыки работы со структурами данных и анализа производительности программ.